# AnalysisTools User Manual

RNDr. Karel Šindelka, Ph.D.
k.sindelka@gmail.com

Version 4.x (2nd June 2026)

# Contents

# 1. Introduction

AnalysisTools is a set of utilities to (mainly) analyse trajectories of particle-based simulations. The utilities (including one from the DLMESO simulation package) can be roughly divided into four types:

- utilities to calculate system-wide properties; e.g., pair correlation functions, densities, or orientational order parameters
- utilities to calculate per-molecule or per-aggregate properties (where aggregate stands for any supramolecular structure); e.g., shape descriptors, persistence lengths, or membrane properties such as bending rigidity and tail interdigitation
- utilities to manipulate or generate configurations; e.g., create initial configurations from scratch, add molecules to an existing system, or join two systems together
- helper utilities to analyse text files; e.g., calculate averages and standard deviations of a data series

The `Examples` directory contains examples showcasing capabilities of individual utilities.

## 1.1 Installation

Requirements

- `GSL`, the GNU Scientific Library
- `cmake` to generate files necessary to build AnalysisTools
- `C` and `Fortran` compilers

To compile the utilities, run the `compile.sh` script in the `AnalysisTools` root directory. By default, the build directory is `build` in the `AnalysisTools` root; to use a different directory add it as an argument to the script, e.g., `./compile.sh custom_dir`. In both cases, the executables will be located in the `bin/` subdirectory of the build directory. To compile only a single utility, run `make <utility name>` directly from within the build directory.

# 2. Format of input/output files

This chapter describes several file types used by many of the AnalysisTools utilities. Output files for the utilities themselves are described in their respective sections in Utilities (chapter 3).

Input files are divided into two categories: structure and coordinate files. The structure files contain information about the system composition, that is, number of beads and molecules, bead properties (name, mass, charge, etc.), molecule properties (name, number and types of beads, connectivity, etc.). The coordinate files contain individual timesteps with beads' coordinates (and, possibly, velocities, forces, etc.). While most files can be used as both a structure and a coordinate file, using a separate structure file may be essential. For example, an `xyz` file (see below) may be used as both, but because it only provides bead names, using a separate structure file would provide additional information (bead and/or molecule properties).

For working with aggregates, AnalysisTools' `agg` file format (see below) describes their compositions in terms of molecule numbers taken from the structure file used to generate it. The Aggregates (section 3.3) utility creates the `agg` file.

## 2.1 Structure files

The following subsections describe the supported formats; browse the `Examples` directory for example files. Most utilities can load a specific structure file using the `-i <file> [type]` option (with optional `type` provided if it doesn't have expected extension); if the option is not used, the coordinate file is typically used as a structure file as well. The type of the input coordinate file can similarly be overridden using the `-ft <type>` option. Recognised type strings are listed in each subsection below.

Importantly, many of the AnalysisTools utilities work (either fundamentally or optionally) with types of beads and/or molecules. In these cases, names are used specify the bead/molecule types. While many input files support naming beads and molecules, not all do (in some formats, these are optional); in such cases, AnalysisTools names the bead types as `b0`, `b1`, etc., up to `bN` where `N` is the number of bead types and molecule types as `m0`, `m1`, etc. If unsure, the Info (section 3.8) utility can provide the names of beads and molecules for the specific structure file.

Info utility can also convert these formats between each other via the `-o <file>` option.

### 2.1.1 VTF format

See here for the complete description of `vtf/vsf` files (a `vtf` file contains a structure part followed by a coordinate part, while a `vsf` file contains only the structure part).

In this format, every bead is defined on its own line (except for the `atom default` bead type) and all bonds all listed.

Not all the keywords listed on the web page for the 'Atom lines' are recognized; keyword `n[ame]` is mandatory and optional keywords are `m[ass]`, `charge|q`, `r[adius]`, `resid`, and `res[name]`. For the `<aid-specifier>`, only a single bead index number or the `default` keyword can be used. Other keywords are ignored.

AnalysisTools groups beads (and molecules) into bead (and molecule) types. Generally, only name defines types of beads, i.e., should two beads share the name, they will be of the same type; their mass, charge, and radius will each equal to that for the bead type's topmost atom line containing the corresponding keyword. If the keyword is missing from all atom lines, that characteristic is undefined. In the case of the Info (section 3.8) utility (see there for details), the `--detailed` command line option triggers the detailed recognition, where beads that share the name but differ in mass, charge, and/or radius will be of different type (with a number appended to the original name).

Different molecule types, on the other hand, are always distinguished based on all their characteristics, i.e., name, connectivity, and order of bead types (order of its `vsf` indices). Molecule types may also be affected by the `--detailed` option because of the bead type definition.

For bond lines, only one bond per line is permitted (i.e., `bond <index1>:<index2>` format).

Periodic boundary condition can be included as `pbc <x> <y> <z>` for a cuboid box or `pbc <a> <b> <c> <alpha> <beta> <gamma>` for a general rhomboid box.

### 2.1.2 FIELD format

A format used by the DL_MESO_DPD simulation package concisely defines species of beads and molecules, providing number of beads/molecules of given species as well as their properties.

The `species` section is the same as in the `DL_MESO FIELD` file, but the `molecules` section differs slightly. Besides the `beads` part, every molecule can have `bonds`, `angles`, and `dihedrals`, but even though up to three parameters for potential parameters are read, AnalysisTools does not recognize the potential keywords (`harm`, `cos`, etc.). Moreover, these potential parameters are only used for generating new `LAMMPS` or `DL_MESO` data files via, e.g., the Info (section 3.8) utility. The `molecules` section can also contain one extra part: `impropers` for improper angles which has the same format as the `dihedrals` section (this was added because the LAMMPS molecular dynamics simulator and other software distinguish between the two types).

Note that the `FIELD` file does not have to contain the `Interactions` section; this part is ignored.

However, using this file in conjunction with coordinate files is not recommended. Because `FIELD` contains only numbers of beads of given types, it is not possible to ensure that bead indices in the coordinate file will line up with the correct bead types in the `FIELD` file. AnalysisTools assigns the bead indices on the 'first read, first labelled' basis, that is, the unbonded beads are first (in the order of the lines in the `species` section) followed by beads from the molecules. Should one want to

check the bead index assignments, the `Info` utility can be used to generate, e.g., a `vsf` file from the `FIELD` file.

Nevertheless, the file can be used to supply extra information for bead and molecule types for the `Info` utility's `-i` option as these do not depend on individual beads but rather on bead and molecule type names (see Info (section 3.8) for details). Moreover, `FIELD` is used as an input file to add extra species into an existing system or to create a new system from scratch via, e.g., AddToSystem (section 3.1).

To be recognized by AnalysisTools utilities, the file must either be called `FIELD` or have the `.field` extension (case-insensitive); alternatively, the type can be specified explicitly with `-i <file> field`.

### 2.1.3 XYZ format

This is the simplest and probably best known format, see, e.g., here for the description. An `xyz` file is essentially a file with timesteps (bead coordinates), and the only structure information are bead names. Note that the second line of each timestep (a comment line) is ignored.

When used as a structure file, the first timestep is used to define the system composition. Therefore while the timesteps can each have a different number of beads, their amount cannot exceed the number in the first step.

### 2.1.4 LAMMPS data format

This rather complicated format comes from the LAMMPS molecular dynamics simulator and lists the numbers of beads, bonds, angles, etc., as well as individual beads, bonds, angles, etc., and information about potentials for bonds, angles, etc. See here for description of the format and the `Examples` directory for example files.

AnalysisTools reads the header information as well as `Masses`, `Atoms`, `Velocities`, `Bonds`, `Angles`, `Dihedrals`, `Impropers`. It also reads `Bond Coeffs`, `Angle Coeffs`, `Dihedral Coeffs`, and `Improper Coeffs` sections.

The `Atoms` section may have `atom_style full`, `bond`, `angle`, `atomic`, `molecular`, or `charge` format (specified as a trailing comment on the `Atoms` line as required by the `LAMMPS` software).

For the `Bond Coeffs` and `Angle Coeffs` sections, harmonic potential is assumed, and the first value of each bond/angle type is multiplied by 2 because LAMMPS uses harmonic spring strength of $k/2$ (as opposed to, e.g., DL_MESO which uses $k$). For the `Dihedral Coeffs` and `Improper Coeffs` sections, AnalysisTools reads at most three numbers without assuming any specific potential. The `Coeffs` sections are used only for generating new structure files via, e.g., the Info (section 3.8) utility; note that to run `LAMMPS` simulations, these section may have to be manually corrected to conform to the specific potential type.

To be recognized by AnalysisTools utilities, the file must have the `.data` extension; alternatively, use `-i <file> data` for a structure file or `-ft data` for a coordinate file.

### 2.1.5 LAMMPS lammpstrj format

This file format comes from the `LAMMPS` dump style custom command.

Of the possible `LAMMPS` attributes, AnalysisTools recognizes `id` (bead index – mandatory), `element` (bead name), `x y z` (Cartesian bead coordinates), `xs ys zs` and `xsu ysu zsu` (scaled coordinates, automatically converted to Cartesian), `xu yu zu` (unwrapped Cartesian coordinates), `vx vy vz` (bead velocities), `fx fy fz` (bead forces), `type` (bead type used as a fallback name when `element` is absent), and `q` (bead charge). Other attributes are ignored. Note that if `element` is missing, the `type` attribute is used as the bead name; if both are absent, all beads are considered of the same type called `b0`.

When used as a structure file, the first timestep is used to define the system composition. Therefore while the timesteps can each have a different number of beads, their amount cannot exceed the number in the first step.

To be recognized by AnalysisTools utilities, the file must have the `.lammpstrj` extension; alternatively, use `-i <file> ltrj` (or `lammpstrj`) for a structure file or `-ft ltrj` for a coordinate file.

### 2.1.6 DL_MESO CONFIG format

A coordinate file format used by the DL_MESO_DPD simulation package. The file starts with a title line, a line with two integers (key and imcon, where imcon specifies the periodic boundary condition type), and three lines defining the simulation box vectors. Each subsequent bead entry consists of a name-and-index line followed by a line with Cartesian coordinates.

Currently, AnalysisTools can write `CONFIG` files but not read them; it is therefore available as an output format only.

To be recognized as an output format, the file must be named `CONFIG` or have the `.config` extension (case-insensitive).

### 2.1.7 GROMACS ITP format

A topology file format used by the GROMACS molecular dynamics software. AnalysisTools reads the `[ moleculetype ]`, `[ atoms ]`, `[ bonds ]`, `[ angles ]`, and `[ dihedrals ]` sections. As `itp` files contain no coordinates, this format can only be used as a structure file.

To be recognized by AnalysisTools utilities, the file must have the `.itp` extension; alternatively, the type can be specified with `-i <file> itp`.

### 2.1.8 PDB format

The Protein Data Bank format is widely used by molecular simulation and visualisation programs. AnalysisTools reads the `CRYST*` record for the box dimensions (only the first three values, i.e. the box edge lengths; an orthogonal box is assumed) and `ATOM` records for bead information: the atom name field gives the bead type, the residue name field gives the molecule type, and the coordinate fields give the bead

position. HETATM and other records are ignored. As pdb files contain no topology, this format can only be used as a structure file.

To be recognized by AnalysisTools utilities, the file must have the .pdb extension; alternatively, the type can be specified with -i <file> pdb.

## 2.2 Coordinate files

As a coordinate file, any of the following formats (described above) can be used:

- vtf/vcf format (vtf file contains both structure and coordinate sections while vcf file contains only the coordinate section); both indexed and ordered timesteps can be used, but the AnalysisTools utilities always output files with indexed timesteps
- xyz format where timesteps can have a different number of beads
- LAMMPS data format which by definition can only contain one timestep
- LAMMPS lammpstrj format where timesteps can have a different number of beads
- DL_MESO CONFIG format; currently available for output only (reading is not yet supported)

## 2.3 Aggregate file

An agg is generated using any of the Aggregates utilities. The file contains information about the number of aggregates in each timestep and which molecules belong to which aggregate. It serves as an additional input file for utilities that calculate aggregate properties; agg file is, therefore, linked to the structure and coordinate file(s) used to generate it.

The agg file is a simple text file. The first two lines are just comments (the second one should contain the command used to generate the file as parts of the command may be used by subsequent aggregate analysis). Starting at the third line, the data for individual timesteps are shown. It follows these rules:

- each timestep starts with Step: <int>
- the second line is the number of aggregates in the given timestep
- for each aggregate, there are *two* consecutive lines:
  - <nCore> : <id1> <id2> ... — the number of core molecules followed by their molecular indices (from the input structure file)
  - <nBorder> : <id1> <id2> ... — the number of border molecules followed by their molecular indices

  Core and border molecules are the result of the DBSCAN algorithm used to identify aggregates; free molecules (monomers) appear as singletons with one core molecule and no border molecules
- no blank or comment lines are allowed
- not all molecules present in the coordinate/structure file(s) used to generate this file must be present in every timestep
- the line Last Step: <int> signalizes the end of data (only the first word Last is checked)

Note that the term aggregate also refers to free molecules (i.e., unimers or fully dissolved chains).

If `vtf` format is used for the coordinates, the indices from `agg` file can be used in vmd to visualize, e.g., only a specific aggregate by using `resid <id1> ... <id\#>` in the `Selected Atoms` box inside the vmd. A rough script to visualize aggregates in different colours is in the `Examples/scripts/Visualize` directory (`VisAgg*` files), accompanied by the resultant picture (the bash script uses the provided `vtf` and `agg` files to generate a `tcl` script and run it via `vmd`).

An example of an `agg` file can be found in the `Examples/AggNumber` directory.

## 2.4 System info file

A system info file maps names to the molecule types present in a system. It is used with the `-sys` option in utilities such as AddToSystem (section 3.1) and Info (section 3.8) when molecule names are not available from the coordinate file.

The file is plain text with one line per molecule type (in the order they appear in the system), each formatted as a name followed by an integer count. Blank lines and comment lines (beginning with `#`) are ignored. The count value is not used when reading — it is only required for the line to be recognised — but when written by a utility, names are left-padded to 20 characters and counts to 7 digits. A warning is issued if the number of valid entries does not match the number of molecule types in the system.

## 2.5 Library directory

A library directory is specified via the `-lib` option (e.g. in AddToSystem (section 3.1) and Info (section 3.8)) and provides pre-defined bead types, interaction parameters, and molecule templates. In all library files, blank lines and comment lines (beginning with `#`) are ignored; the first non-comment, non-blank line in each index file is a count that is skipped.

The directory must contain four index files:

- `list_parameters.txt` — bead types and self-interactions; data lines have the form `<index> <name> <mass> <charge> <A> <Rc>`, where `A` is the DPD repulsion amplitude and `Rc` is the cutoff radius.
- `list_bonds.txt` — harmonic bond types; data lines have the form `<bond_ID> <k> <r0>`, where `k` is the spring constant and `r0` is the equilibrium length. Fortran-style `d`-exponent notation (e.g. `1.5d0`) is accepted for `r0`.
- `list_angles.txt` — harmonic angle types; data lines have the form `<angle_ID> <k> <theta>`, where `k` is the spring constant and `theta` is the equilibrium angle.
- `list_cross_interactions.txt` — pairwise DPD interactions between different bead types; data lines have the form `<index> <name1> <name2> <A> <Rc>`.

Additionally, each molecule template is stored in its own file `<molname>.txt` within the library directory. The format is similar to the `FIELD` file: the first two non-comment lines are an arbitrary key string (ignored) and the number of beads,

followed by bead lines of the form `<index> <bead_ID> <x> <y> <z>` (bead indices are 1-based). An optional `Bonds` section (the keyword on its own line, followed by a count line, then `<bond_ID> <bead_i> <bead_j>` lines) and an optional `Angles` section (same structure, with `<angle_ID> <bead_i> <bead_j> <bead_k>` lines) may follow. The file is terminated by `End`. Molecule templates are loaded by name via the `-mol` option.

# 3. Utilities

All utilities have command line options affecting their behaviour. The following options are common for many utilities, so they are described here rather than at the individual utilities where only list of the possible options is provided.

| General options | |
|---|---|
| `--verbose` | print system composition (bead and molecule types, counts, bond/angle/dihedral types, and box dimensions) to the screen |
| `--silent` | suppress all progress and informational output (overrides `--verbose`; errors and warnings are still shown) |
| `--help` | print short description of the utility and exit |
| `--version` | print version of the utilities and exit |
| **Options for structure and coordinate input files** | |
| `-i <stru> [type]` | use specified structure file; optional `type` overrides extension-based format detection, see Structure files (section 2.1) for possible formats and type strings |
| `-ft <type>` | override input file type: `vtf`, `xyz`, `data`, `ltrj` (or `lammpstrj`), `field`, `itp`, `pdb` |
| `-st <n>` | start calculations at `n`-th timestep |
| `-e <n>` | end calculations at `n`-th timestep |
| `-sk <n>` | skip every `n` timesteps (i.e., use every `(n+1)`-th timestep) |

The `-st`, `-e`, and `-sk` options can be combined, so that, for example, specifying `-st 2 -e 10 -sk 2` would use timesteps 2, 5, and 8.

Should two (or more) identical options be specified on the command line, only the first one is used.

Note that while in theory it is possible to combine any coordinate and structure file formats via the `-i` option, it is generally not recommended. For example, `xyz` file format does not include bead indices which means that beads are numbered according to the file line which may not agree with `vsf` structure format that does contain bead indices.

## 3.1 AddToSystem

This utility takes an existing system (or creates a new one from scratch) and adds new beads (read from a `FIELD` file) into it, placing them either completely randomly or according to supplied constraints.

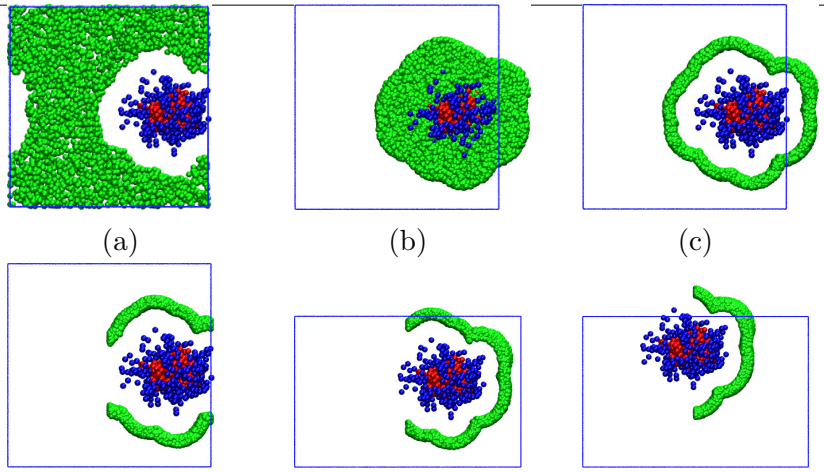



Figure 3.1: Examples of distance checks: (a) `-ld`, (b) `-hd`, and (c) both `-ld` and `-hd`. Combining the `-ld` and `-hd` options with (d) `-cx` axis constraint, (e) `-cx` and box-size-defining `-b` options, and (f) `-cx`, `-b`, and offsetting `-off` options. The `-bt` and `-add` options were used everywhere. Colours: red and blue – original system, green – added beads, blue line – new simulation box. Note that the pictures show cross-sections of the simulation box to highlight relative position of the original and added beads. See the `01-ManualExamples` directory for exact commands.

There are two types of constraints which can be combined: place new beads (i) specified distance from other beads or (ii) in a given interval in x-, y-, and/or z-axis directions. In (i), options `-ld` and/or `-hd` define the lowest and/or highest distance from beads specified by `-bt <name(s)>` (bead name) or `--bonded` (all bonded beads). In (ii), options `-cx`, `-cy`, and `-cz` basically change the box size for the added beads. By default, this constraint is specified in units relative to the box dimensions (i.e., allowed values go from 0 to 1); use the `--real` option for absolute units. For example, `-cx 0.5 1` would generate x coordinates between 50% and 100% of the box's x sidelength.

All these options can be combined, but note that `AddToSystem` does not perform any sanity checks; that is, if any combination of the provided options is impossible to achieve, the utility may run forever. See figure 3.1 for schematic examples and the `01-ManualExamples` directory for the exact commands to produce these examples.

When adding molecules, either the molecule's geometric centre (default behaviour) or its first bead (`--head` option) obey these constraints. The coordinates of the remaining beads in the molecule are governed by the coordinates in the input `FIELD` file. Therefore, not all the molecular beads necessarily obey the constraining rules.

By default, molecules are added with a random orientation; when `--no-rotate` switch is used, the molecules are not rotated, but when `-a` option is used, all the molecules are rotated by the three angles specified in degrees. The three angles in `-a <$\alpha $> <$\beta $> <$\gamma $>` specify rotations around the $x$-, $y$-,

and $z$-axes by $\alpha$, $\beta$, and $\gamma$ degrees, respectively. The rotation matrix $R$ is

$$R = \begin{bmatrix} \cos\gamma\cos\beta & \sin\gamma\cos\beta & -\sin\beta \\ \cos\gamma\sin\beta\sin\alpha - \sin\gamma\cos\alpha & \sin\gamma\sin\beta\sin\alpha + \cos\gamma\cos\alpha & \cos\beta\sin\alpha \\ \cos\gamma\sin\beta\cos\alpha + \sin\gamma\sin\alpha & \sin\gamma\sin\beta\cos\alpha - \cos\gamma\sin\alpha & \cos\beta\cos\alpha \end{bmatrix} \tag{3.1}$$

By default, all new beads are exchanged for beads in the original system; what bead type to exchange is either the most numerous one (default behaviour) or the name(s) provided via the `-xb` option. Note that the utility doesn't check whether exchanged beads are in a molecule, so a bonded bead may be exchanged, leaving only part of the molecule in the new system. For adding beads into the system rather than exchanging them, use the `--add` option.

The size of the new simulation box can be changed using the `-b` option (the new size must be in absolute units, i.e., it is unaffected by the `--real` option). Any constraints for placing beads are applied to this box rather than the one in the original system. By default, the new system is placed so the centres of the original box and the new one coincide. Note that periodic boundary conditions are not applied to the new system, so the beads can be outside the simulation box. If `-off` option is used, the beads from the input system are instead moved by the offset specified by the fraction of the output simulation box (or in real units, if `--real` is used) before the new beads are added.

To create a new system instead of adding to an existing one, use `-` instead of an `<input>` file. An extra file can be generated via the `-o` option; this can be useful to, e.g., generate both `vtf` file for visualization and `data` file for use by LAMMPS.

Examples of using the utility (such as how to generate a bilayer or wire-like aggregate) are provided in the `Examples/AddToSystem` folder (including those in figure 3.1).

Usage: `AddToSystem <input> [<in.field>] <output> [options]`

| Mandatory arguments | |
|---|---|
| `<input>` / `-` | input coordinate file, or `-` to create a system from scratch |
| `[<in.field>]` | `FIELD` file defining species to add (optional when `-lib` is used) |
| `<output>` | output coordinate file |

| Options | |
|---|---|
| `-o <file>` | extra output structure file |
| `-lib <dir>` | library directory; load molecules via `-mol` instead of `<in.field>` |
| `-mol <mol> <\%/n> ...` | molecule name(s) from the library with a fraction (e.g. `5\%`) or integer count; pairs can be repeated |
| `-sys <input> [out]` | system info file to assign names to existing molecule types; optionally write updated info |

| | |
|---|---|
| `-ld <float>` | lowest distance from beads specified by `-bt` or `--bonded` |
| `-hd <float>` | highest distance from beads specified by `-bt` or `--bonded` |
| `-bt <name(s)>` | bead types to use with `-ld`/`-hd` |
| `--bonded` | use all bonded beads for `-ld`/`-hd` (overwrites `-bt`) |
| `-xb <name>` | bead type to exchange (default: most numerous type) |
| `--add` | add new beads to the system instead of exchanging existing ones (overwrites `-xb`) |
| `--no-rotate` | do not randomly rotate added molecules |
| `-a 3x<angle>` | rotate all added molecules by the three given angles (degrees); overrides `--no-rotate` |
| `--head` | use molecule's first bead for constraint checks (overrides `--tail`) |
| `--tail` | use molecule's last bead for constraint checks (default: geometric centre) |
| `-cx <lo> <hi>` | constrain x coordinate to interval $\langle lo, hi)$ |
| `-cy <lo> <hi>` | constrain y coordinate to interval $\langle lo, hi)$ |
| `-cz <lo> <hi>` | constrain z coordinate to interval $\langle lo, hi)$ |
| `--real` | use real units for `-cx`/`-cy`/`-cz`/ `-off` instead of fractions of the output box size |
| `-b <x> <y> <z>` | side lengths of the new simulation box (real units) |
| `-off <x> <y> <z>` | offset the original system by the given amount (fractions of output box, or real units with `--real`) |
| `-ntot <n> <type>` | fill to `n` total beads using `<type>` from the library (requires `-lib`) |
| `-ebt <int>` | number of extra bead types to reserve in LAMMPS data file output |
| `-s <int>` | seed for random number generator |
| `-i`, `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version` | |

## 3.2 AggNumber

This utility can calculate time evolution of average aggregation sizes, distributions of aggregate sizes, and/or compositional properties of aggregates with specified aggregate sizes from a supplied `agg` file (see Section 2.3 for its format).

Generally, for a quantity $\mathcal{O}$, the number, weight, and z averages, $\langle\mathcal{O}\rangle_\mathrm{n}$, $\langle\mathcal{O}\rangle_\mathrm{w}$, and $\langle\mathcal{O}\rangle_\mathrm{z}$, respectively, are defined as

$$\langle\mathcal{O}\rangle_\mathrm{n} = \frac{\sum_i N_i\mathcal{O}_i}{N}, \quad \langle\mathcal{O}\rangle_\mathrm{w} = \frac{\sum_i N_i m_i\mathcal{O}_i}{\sum_i N_i m_i}, \text{ and } \quad \langle\mathcal{O}\rangle_\mathrm{z} = \frac{\sum_i N_i m_i^2\mathcal{O}_i}{\sum_i N_i m_i^2}, \tag{3.2}$$

where $N$ is the total number of measurements, i.e., the total number of aggregates for per-aggregate averages (or molecules for per-molecule averages); $N_i$ is the number

of measurements with the value $\mathcal{O}_i$, and $m_i$ is mass of an aggregate $i$ (or a molecule $i$).

Number, weight, and z distribution functions of aggregate sizes, $F_{\mathrm{n}}(A_{\mathrm{S}})$, $F_{\mathrm{w}}(A_{\mathrm{S}})$, and $F_{\mathrm{z}}(A_{\mathrm{S}})$, respectively, are defined as

$$\begin{aligned} F_{\mathrm{n}}(A_{\mathrm{S}}) &= \frac{N_{A_{\mathrm{S}}}}{\sum_{A_{\mathrm{S}}} N_i} = \frac{N_{A_{\mathrm{S}}}}{N}, \\ F_{\mathrm{w}}(A_{\mathrm{S}}) &= \frac{N_{A_{\mathrm{S}}} m_{A_{\mathrm{S}}}}{\sum_{A_{\mathrm{S}}} N_i m_i} = \frac{N_{A_{\mathrm{S}}} m_{A_{\mathrm{S}}}}{\sum_{i=1}^{N} m_i} = \frac{N_{A_{\mathrm{S}}} m_{A_{\mathrm{S}}}}{M}, \text{ and} \\ F_{\mathrm{z}}(A_{\mathrm{S}}) &= \frac{N_{A_{\mathrm{S}}} m_{A_{\mathrm{S}}}^2}{\sum_{A_{\mathrm{S}}} N_i m_i^2} = \frac{N_{A_{\mathrm{S}}} m_{A_{\mathrm{S}}}^2}{\sum_{i=1}^{N} m_i^2}, \end{aligned} \tag{3.3}$$

where $N_{A_{\mathrm{S}}}$ and $m_{A_{\mathrm{S}}}$ stand for the number and mass, respectively, of aggregates with aggregate size $A_{\mathrm{S}}$; $M$ is the total mass of all aggregates. The equations are normalized so that $\sum F_x(A_{\mathrm{S}}) = 1$.

Per-timestep averages and/or distributions are calculated when `-a` and/or `-d` are used, respectively. Overall averages are appended to the end of the file(s). Distributions of numbers of molecules of different types in aggregates of specified sizes are calculated when `-c` option is used. In this case, two files are generated for every specified aggregate size: first, a number distribution of each molecule type in the aggregates; second, a number distribution of ratios of all possible molecule type pairs in the aggregates, which can be plotted as 3D graph, providing information useful for analysing aggregates comprising multiple molecule types.

Note that at least one of the `-a`, `-d`, and `-c` options must be specified in the `AggNumber` command. Examples for all three options, are shown in the analysis of a sandbox system in `Examples/SandboxSystem/Analysis.pdf`.

The definition of aggregate size is flexible. If none of `-m`, `-x`, or `-only` options is used, aggregate size is the 'true' aggregation number, i.e., the number of all molecules in the aggregate; if `-m` is used, aggregate size is the sum of only specified molecule type(s); if `-x` is used, aggregates containing only specified molecule type(s) are disregarded; if `-only` is used, only aggregates composed of the specified molecule type(s) are taken into account. These options can be combined. For example, consider a system containing three aggregates composed of various numbers of three different molecule types:

| Molecule types | Aggregate composition |
|---|---|
| `Mol_A` | `Agg_1`: 1 `Mol_A` +2 `Mol_B` +3 `Mol_C` = 6 molecules |
| `Mol_B` | `Agg_2`: 1 `Mol_A` +2 `Mol_B` = 3 molecules |
| `Mol_C` | `Agg_3`: 1 `Mol_A` = 1 molecule |

Following is a table of some of the possibilities depending on the option(s) used:

| Used options | Aggregates Detected ($A_{\mathrm{S}}$) | Ignored |
|---|---|---|
| | `Agg_1` (6) | |
| (1) — | — | |

| | | |
|---|---|---|
| | Agg_2 (3)<br>Agg_3 (1) | |
| (2) -m Mol_A Mol_B | Agg_1 (3)<br>Agg_2 (3)<br>Agg_3 (1) | — |
| (3) -m Mol_B Mol_C | Agg_1 (5)<br>Agg_2 (2) | Agg_3 |
| (4) -x Mol_A Mol_B | Agg_1 (6) | Agg_2<br>Agg_3 |
| (5) -x Mol_A Mol_B -m Mol_A Mol_B | Agg_1 (3) | Agg_2<br>Agg_3 |
| (6) -only Mol_A Mol_B | Agg_2 (3)<br>Agg_3 (1) | Agg_1 |
| (7) -only Mol_A Mol_B -m Mol_A | Agg_2 (1)<br>Agg_3 (1) | Agg_1 |
| (8) -only Mol_A Mol_B -x Mol_A | Agg_2 (3) | Agg_1<br>Agg_3 |
| (9) -only Mol_A Mol_B -x Mol_A -m Mol_A | Agg_2 (1) | Agg_1<br>Agg_3 |

Note that when the -m option is used, there are two possible definitions of aggregate mass, $m_{\mathrm{agg}}$; e.g., in (9), the one detected aggregate may have 'partial mass' consisting of the selected molecule types only ($m_{\mathrm{agg}} = m_{\mathtt{Mol\_A}}$) or 'total mass' ($m_{\mathrm{agg}} = m_{\mathtt{Mol\_A}} + 2m_{\mathtt{Mol\_B}}$). For weight and z average aggregate sizes, $\langle A_{\mathrm{S}} \rangle_{\mathrm{w}}$ and $\langle A_{\mathrm{S}} \rangle_{\mathrm{z}}$, respectively, and their distributions, $F(A_{\mathrm{S}})_{\mathrm{w}}$ and $F(A_{\mathrm{S}})_{\mathrm{z}}$, the different mass definitions lead to different results that can both be physically meaningful, so they are all included with the -a <file> and/or -d <file> options are used. However, for all other variables such as average aggregate mass or shape descriptors of aggregates (calculated via the GyrationAggregates utility) only the total mass of all molecules provides physically meaningful results, so only properties with the total aggregate mass are calculated.

Lastly, only a specified range of aggregate sizes can be taken into account (-n <size> <size> option). These sizes are defined by the -m, -x, and -only options as well.

Usage:

```
AggNumber <in.stru> <in.agg> [options]
```

| Mandatory arguments | |
|---|---|
| <in.stru> | input structure file |
| <in.agg> | input agg file |
| Non-standard options | |

| | |
|---|---|
| `-a <file>` | calculate per-timestep averages |
| `-d <file>` | calculate distribution of aggregate sizes |
| `-c <file> <size(s)>` | calculate composition distribution for specified aggregate size(s), writing to two `<file>`'s with automatic ending `-<size>.txt` and `_r-<size>.txt` |
| `-n <int> <int>` | use aggregate sizes in a given range |
| `-m <name(s)>` | use number of specified molecule(s) as aggregate size |
| `-x <name(s)>` | exclude aggregates containing only specified molecule(s) |
| `-only <name(s)>` | use only aggregates composed of specified molecule(s) |
| `-w <width>` | width of a single bin for the ratios in `-c` (default: 0.1) |

Other options (see the beginning of Chapter 3)

`-st`, `-e`, `-sk`, `--help`, `--verbose`, `--silent`, `--version`

Format of output files:

1) `-d <file>` – distributions of aggregate sizes
   - first two lines: AnalysisTools version and the command
   - third line: column headers
     - first is the aggregate size, `As`; either true aggregation number or the size specified via the `-m`, `-x`, `-only`, and/or `-n` options
     - `F_n(As)`, `F_w(As)`, and `F_z(As)` are number, weight, and z distribution of aggregate sizes (Equation (3.3))
       - for both `F_w(As)` and `F_z(As)`, two columns are provided with the two definitions of aggregate mass ('partial mass' is the mass of `-m`-option-defined molecules only); if `-m` option is not used, the two data columns are identical
     - next is the total number of aggregates with specified size (sum from all timesteps)
     - the remaining columns (`<mol name>_n`) show average numbers of every molecule type in an aggregate with the specified size
   - fourth line: note explaining the two mass definitions
   - data lines follow
   - second to last line: column headers for overall averages
     - `<As>_n`, `<As>_w`, and `<As>_z` are number, weight, and z averages, respectively, of aggregate numbers (see Equation (3.2) for definitions)
       - similarly to `F_w(As)` and `F_z(As)`, two data points are provided for each of `<As>_w` and `<As>_z` for the two mass definitions
     - `<M>_n`, `<M>_w`, and `<M>_z` are number, weight, and z averages, respectively, of aggregate masses (see Equation (3.2) for definitions)
     - next are average numbers of every molecule type in an aggregate with the specified size (`<mol name>_n`)
     - average number of aggregates per timestep, `<n_agg>`
   - last line: the overall averages

2) `-a <file>` – per-timestep averages
   - first two lines: AnalysisTools version and the command
   - third line: column headers

  - first is simulation timestep
  - the calculated data follow: number, weight, and z average aggregate size (`<As>_n`, `<As>_w`, and `<As>_z`, respectively) and mass (`<M>_n`, `<M>_w`, and `<M>_z`, respectively)
    - similarly to `-d <file>` case, two columns are provided for each of `<As>_w` and `<As>_z` for the two mass definitions
  - the next columns (`<mol name>_n`) show average numbers of every molecule type per aggregate
    - note that on each data line, the sum all these columns equals number average aggregate size, the `<As>_n` column
  - the last column is the number of aggregates in the given step
- fourth line: note explaining the two mass definitions
- data lines follow
- the last two lines are the same as in `-d <file>`

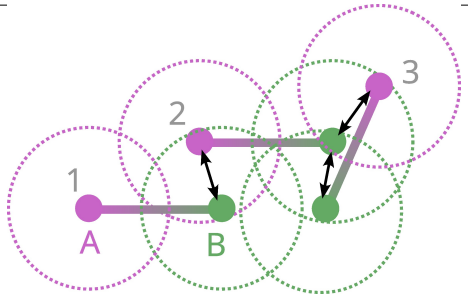
3) `<file>-<size>.txt` from `-c` option – composition distribution of numbers of molecules
- first two lines: AnalysisTools version and the command
- third line: number of aggregates of given size (sum from all timesteps)
- fourth line: column headers
  - first is the number of molecules in the aggregate
  - the rest are the number distributions of for each molecule type in the given aggregate size
- data lines follow

4) `<file>_r-<size>.txt` from `-c` option – composition distribution of ratios of molecular pairs
- first two lines: AnalysisTools version and the command
- third line: number of aggregates of given size (sum from all timesteps)
- fourth line: column headers
  - first two columns are numbers of `mol_A` and `mol_B` molecules in the aggregate
  - the rest are the distributions for `mol_A` vs `mol_B` molecule types (one column for each possible molecule type pair)
- data lines follow

## 3.3 Aggregates

These utilities determine which molecules belong to which aggregates (note that 'aggregate' is used for any supramolecular structure) according to a simple criterion: two molecules belong to the same aggregate if they share at least a specified number of contact pairs. A contact pair is a pair of two beads belonging to different molecules which are closer than a specified distance. The information is written into an `.agg` text file described in Section 2.3.

Periodic boundary conditions can be removed from whole aggregates via the `-j` option; the aggregate's centre of mass is always inside the simulation box. This is useful for visualization (an aggregate will no longer be split by the walls of the simulation box, but it may stretch far outside the boundaries of the box) as well as for further analysis (the other utilities do not have to join the aggregates, possibly




`Aggregates ... A B`
⇒ 3 contact pairs
`Aggregates-NotSameBeads ... A B`
⇒ 2 contact pairs

Figure 3.2: Simple example of distance checks—three two-bead molecules, where dotted lines represent maximum pair distance (`-d` option) and black arrows show contact pairs. Note the same bead can be a part of multiple pairs.

greatly speeding up subsequent analyses).

While the `Aggregates` utility uses all possible pairs of given bead types, `Aggregates-NotSameBeads` does not use same-type pairs. For example, if bead types `A` and `B` are given, `Aggregates` will use all three possible bead type pairs (i.e., `A-A`, `A-B`, and `B-B`), but `Aggregates-NotSameBeads` will use only `A-B` bead type pairs. Fig 3.2 illustrates the behaviour: using `Aggregates ... A B` would find one contact pair between molecules 1 and 2 and two contacts between molecules 2 and 3, whereas `Aggregates-NotSameBeads ... A B` would ignore A-A and B-B pairs, finding no contact pair between molecules 1 and 2 and only one between molecules 2 and 3.

If `-w` option is used, the utilities also take the found aggregates, testing which are touching the specified wall(s) and saving those touching the wall(s) and those in bulk into two extra `.agg` files. The wall(s) are specified via the `<a>` argument, which defines the axis perpendicular to the wall (the argument must be `x`, `y`, or `z`), and numeric argument(s), which define the coordinate(s) of the wall(s) along the axis. For example, assuming a simulation box with size $10^3$ and a slit configuration with two walls in the xy-plane 1 distance unit from the box edges, then `-w z 1 9` would be used. The distance for checking whether a bead touches the wall is the same as the one for determining bead contact pairs. Aggregate data are saved into two extra `.agg` files with names based on the main output `<out.agg>` – `-w` and `-no_w` is placed before the `.agg` extension.

Note that when `-st`, `-e`, or `-sk` options are used, the resulting `agg` file is no longer coupled to the original coordinate file (i.e., they cannot be used together for further analyses), but it is coupled to the optional output file with joined coordinates (`-j` option).

Usage: `Aggregates <input> <out.agg> <bead type(s)> [options]`

| Mandatory arguments | |
|---|---|
| `<input>` | input coordinate file |
| `<out.agg>` | output `agg` file |
| **Options** | |
| `-d <num>` | maximum distance for contact (default: 1) |
| `-c <num>` | minimum number of contacts (default: 1) |

| | |
|---|---|
| `-j <file>` | save coordinates of joined aggregate to a coordinate file |
| Other options (see the beginning of Chapter 3) | |
| `-st`, `-e`, `-sk`, `-i`, `--verbose`, `--silent`, `--help`, `--version` | |

## 3.4 Angle Molecules

This utility calculates angles (distribution and overall averages) between beads in each molecule of specified molecule type(s); the input structure file must contain angles (such as LAMMPS `data` file). It can also calculate angles between any three beads in the molecule type(s) via the `-n` option. See `Examples/AngleMolecules` for an example.

By default, the angles are calculated for every molecule type, but the `-m` option can be used to specify only some of them.

The utility groups angles according to bead types; e.g., in a linear 6-bead molecule (such as in the `Example/AngleMolecules` directory) with bead order A-A-A-B-A-A with all possible angles defined (i.e., 1-2-3, 2-3-4, 3-4-5, and 4-5-6 where the numbers specify bead indices in the molecule), there are three angle types: A-A-A, A-A-B, and A-B-A. However, using the `--all` option, all molecular angles are averaged separately as well; i.e., beside for the three angles based on bead types, the distributions and averages are calculated for all five angles in the molecule too.

Using `-n` option, extra angles are specified by three bead indices taken from the bead order in the molecule type(s). These indices go from 1 to $N$, where $N$ is the number of beads in the molecule type (`Info` utility can be used to check the indices). Multiple angles may be specified, and the results are written to the specified file. Angles containing an index number that is higher than the number of beads in a molecule are ignored only for that molecule.

Usage: `AngleMolecules <input> <width> <output> [options]`

| Mandatory arguments | |
|---|---|
| `<input>` | input coordinate file |
| `<width>` | width of each distribution bin |
| `<output>` | output file for distribution |
| **Options** | |
| `-m <name(s)>` | specify molecule type(s) to use |
| `--joined` | specify that `<input>` contains joined coordinates |
| `--all` | write all angles for the molecule types |
| `-n <file> <ints>` | multiple of three indices for extra angle |
| Other options (see the beginning of Chapter 3) | |
| `-st`, `-e`, `-sk`, `-i`, `--verbose`, `--silent`, `--help`, `--version` | |

Format of output files:

1) `<output>` – distribution of angles (grouped by bead types and possibly for all molecular angles as well if `--all` option is used)
   - first line: AnalysisTools version
   - second line: command used to generate the file
   - following lines (number of specified molecules plus 1): column headers
     - first column is the centre of each bin in angles (governed by `<width>`); i.e., if `<width>` is 5°, then the centre of bin 0 to 5° is 2.5, centre of bin 5 to 10° is 7.5 and so on
     - the rest are for the calculated data: for each specified molecule there are angles grouped by bead types followed by all angles in that molecule (if `--all` option is used)
   - next lines: the calculated data
   - last several lines (number of specified molecules plus 2): minimum, maximum, and arithmetic mean for each calculated angle (last line) and headers (preceding lines)
2) `-n <file>` – angles specified by supplied molecular indices
   - the file structure is the same as for the default output file

## 3.5 Average

This utility calculates simple average and standard error from specified column(s) of data in the input text file (all `\#`-starting lines and blank lines are ignored), printing it to standard output. It can also calculate one of three types of averages based on a used option – an overall average with statistical error and an autocorrelation time estimate (`-tau` option), block averages (`-b` option), or moving averages (`-m` option). While the `-tau` option appends a single line to the output file, either of the `-b` or `-m` options creates a new output file with somewhat smoother data.

The first and last lines used for the average calculation can be controlled using the standard `-st` and `-e` options.

### 3.5.1 Estimate autocorrelation time via -tau option

The average value of an observable $\mathcal{O}$ is a simple arithmetic mean,

$$\langle \mathcal{O} \rangle = \frac{1}{N} \sum_{i=1}^{N} \mathcal{O}_i, \tag{3.4}$$

where $N$ is the number of measurements and the subscript $i$ denotes individual measurements. If the measurements are independent (i.e., uncorrelated), the statistical error, $\epsilon$, is given by:

$$\epsilon^2 = \frac{\sigma^2_{\mathcal{O}_i}}{N}, \tag{3.5}$$

where $\sigma^2_{\mathcal{O}_i}$ is the variance of the individual measurements,

$$\sigma^2_{\mathcal{O}_i} = \frac{1}{N-1} \sum_{i=1}^{N} (\mathcal{O}_i - \langle \mathcal{O} \rangle)^2. \tag{3.6}$$

For correlated data, the autocorrelation time, $\tau$, representing the number of steps between two uncorrelated measurements must be determined. Every $\tau$-th measurement is uncorrelated, so the equation (3.5) can then be used to estimate the error.

A commonly used method to estimate $\tau$ is the binning (or block) method. In this method, the correlated data are divided into $N_\mathrm{B}$ non-overlapping blocks of size $k$ ($N = kN_\mathrm{B}$) with per-block averages, $\mathcal{O}_{\mathrm{B},n}$, defined as:

$$\mathcal{O}_{\mathrm{B},n} = \frac{1}{k} \sum_{\substack{i=1+\\(n-1)k}}^{kn} \mathcal{O}_i. \tag{3.7}$$

If $k \gg \tau$, the blocks are assumed to be uncorrelated and equation (3.5) can be used:

$$\epsilon^2 = \frac{\sigma_\mathrm{B}^2}{N_\mathrm{B}} = \frac{1}{N_\mathrm{B}(N_\mathrm{B}-1)} \sum_{n=1}^{N_\mathrm{B}} (\mathcal{O}_{\mathrm{B},n} - \overline{\mathcal{O}})^2. \tag{3.8}$$

An estimate of the autocorrelation time can be obtained using the following formula:

$$\tau_\mathcal{O} = \frac{k\sigma_\mathrm{B}^2}{2\sigma_{\mathcal{O}_i}^2}. \tag{3.9}$$

The number of blocks, $N_\mathrm{B}$, is supplied as an argument of the `-tau` option and Average then appends a single line to the `<output>` file; the line starts with $N_\mathrm{B}$ and continues with three values ($\langle\mathcal{O}\rangle$, $\epsilon$, and $\tau_\mathcal{O}$) per every data column specified in the Average command.

A way to quickly get a $\tau$ estimate is to use a wide range of $N_\mathrm{B}$ values and plot $\tau_\mathcal{O}$ from equation (3.9) as a function of $N_\mathrm{B}$. Because the number of data points in one block should be significantly larger than the autocorrelation time (e.g., ten times larger), plotting $f(x) = N/(10x)$ will produce a monotonously decreasing curve that intersects the $\tau_\mathcal{O}$ vs. $N_\mathrm{B}$ curve. A value of $\tau_\mathcal{O}$ near the intersection (but to the left, where the decreasing curve is above $\tau_\mathcal{O}$ vs. $N_\mathrm{B}$ curve) can be considered a safe estimate of $\tau$.

### 3.5.2 Block averages via -b option

Besides estimating the autocorrelation time, the per-block averages can be themselves plotted to get (probably) smoother dataset. Using the `-b` option, the number of data points per block to average, $k$, is supplied, and the utility prints the per-block averages from equation (3.7) to the output file.

This way of averaging could be useful for example with density data produced by DensityBox or related utilities; if the bin width supplied to the DensityBox was too small, it is (possibly much) faster to block-average the densities rather than rerun DensityBox. For this case, specify the first column (distance) along with any density columns from the density file.

### 3.5.3 Moving averages via -m option

More common way to smoothing noisy data is to use the moving (or rolling or running) average (or moving mean or rolling mean); this common method does have many names.

Similarly to the block-average, the first element of the moving average is a simple mean of $k$ values. Unlike with block-average, however, the $k$ values for the next element are obtained by ignoring only one value and taking the next $k$ values; i.e., $k-1$ values from the previous element of the moving average are always reused as opposed to the block-average where the blocks of data are not overlapping.

The moving average elements, $\mathcal{O}_{\mathrm{M},n}$, are defined as

$$\mathcal{O}_{\mathrm{M},n} = \frac{1}{k} \sum_{n}^{n+k-1} \mathcal{O}_i. \tag{3.10}$$

Usage: `Average <input> <output> <column(s)> [options]`

| Mandatory arguments | |
|---|---|
| `<input>` | input text file |
| `<output>` | output text file |
| `<column(s)>` | at least one column number from `<input>` |

| Options | |
|---|---|
| `-tau <int>` | $\tau$ estimation where `<int>` represents the number of blocks $N_{\mathrm{B}}$ |
| `-b <int>` | block-average printing mode where `<int>` represents the number of data points per one block, $k$ (equation (3.7)) |
| `-m <int>` | moving average mode where `<int>` represents the number of data points per one moving block, $k$ (equation (3.10)) |

| Other options (see the beginning of Chapter 3) |
|---|
| `-st`, `-e`, `--verbose`, `--silent`, `--help`, `--version` |

## 3.6 BondLength

This utility calculates a distribution of bond lengths in specified molecule type(s). It can also calculate a distribution of distances between any two beads in the molecules.

For each of the specified molecule type(s), `BondLength` calculates bond lengths between all different types of connected bead pairs. For example, assume two linear molecule types `Mol_1` and `Mol_2`, both composed of bead types `A` and `B`. `Mol_1` is connected like this: `A-A-B`; `Mol_2` like this: `A-B-B`. If both molecule types are used, `BondLength` calculates for each molecule type distribution of lengths for bonds `A-A`, `A-B`, and `B-B` (separate for each molecule even though the molecules share the same bead types).

To calculate the distribution of distances between specific (possibly unconnected) beads in a molecule, use `-d` option which takes as arguments pairs of bead indices

(according to the order of beads in the molecule in `vsf` – similarly to the `-n` option in `AngleMolecules`, i.e., Section 3.4). More than one pair can be specified. These indices are the same for all `<mol name(s)>`. If an index higher than the number of beads in the molecule is provided, the utility takes the last bead of the molecule (i.e., the highest index). For example, using `-d file.txt 1 2 1 9999` would write two distributions for each `<mol name(s)>` into `<file.txt>`: distribution of distances between the first and the second bead in each `<mol name(s)>` and between the first bead and the last one (or the 9999th bead).

In both cases, `BondLength` appends at the end of the file minimum and maximum bond lengths/distances.

Usage:

```
BondLength <input> <width> <output> <mol name(s)> <options>
```

| Mandatory arguments | |
|---|---|
| `<input>` | input coordinate file (either `vcf` or `vtf` format) |
| `<width>` | width of each bin of the distribution |
| `<output>` | output file with distribution of bond lengths |
| `<mol name(s)>` | molecule name(s) to calculcate bond lengths for |

| Non-standard options | |
|---|---|
| `-st <int>` | starting timestep for calculation (default: 1) |
| `-e <int>` | ending timestep for calculation (default: none) |
| `-d <out> <ints>` | write distribution of distances between specified beads in each specified molecule to `<out>` |
| `-w <double>` | warn if a bond length exceeds `<double>` (default: half a box length) |

Format of output files:

1) `<output>` – distribution of bond length between all bead pairs
   - first line: command used to generate the file
   - second line: column headers
     - first is the centre of each bin (governed by `<width>`); i.e., if `<width>` is 0.1, then the centre of bin 0 to 0.1 is 0.05, centre of bin 0.1 to 0.2 is 0.15, etc.
     - the rest are for the calculated data: for each molecule type, there is a list of column numbers corresponding to all possible bead type pairs in the molecule; if no beads of the given types are connected, the data column contains `nan`
   - next lines: the calculated data
   - second to last line: column headers for minimal and maximal bond lengths
     - for each molecule, all the possible beads pairs are again listed
     - for each pair, the column number corresponds to the minimum, while the next column is always the maximum
   - last line: commented out (i.e., the line starts with `#`) minimal and maximal bond lengths

2) `-d <output> <ints>` – distribution of distances between specified beads
   - first line: command used to generate the file

- second line: order of beads (given only by bead names) in each molecule
- third line: column headers
  - first is again the centre of every bin
  - the rest are for the calculated data: for each molecule type, there is a list of column numbers corresponding to the given pairs of bead indices in the molecule (and to the `-d` option's arguments); the numbers also correspond to the order of beads in the previous line
- next lines: the calculated data
- second to last line: column headers for minimal and maximal bond lengths
  - for each molecule, all the possible beads pairs are again listed
  - for each pair, the column number corresponds to the minimum, while the next column is always the maximum
- last line: commented out (i.e., the line starts with `#`) minimal and maximal bond lengths

## 3.7 DensityBox

This utility calculates number density for all bead types along all three axis directions of the simulation box, generating one file per axis. The density is calculated from 0 to box length in the given direction, that is, the box is 'sliced' into blocks with width `<width>`, and numbers of different bead types are counted in each 'slice'. Or to put it in other words, a density profile for a bead type $i$ along an axis $\alpha$, $\rho(\alpha)$, is calculated as

$$\rho_i(\alpha) = \frac{\sum_j \delta(\alpha - \alpha_j^i)}{\Delta\alpha L_\beta L_\gamma}, \tag{3.11}$$

where $\delta(\alpha - \alpha_i^j)$ gives the number of $i$ beads inside a slice of the simulation box of the thickness $\Delta\alpha$ (i.e., `<width>`) along the $\alpha$-axis; $L_\beta$ and $L_\gamma$ represent box side lengths along the two remaining axes.

The utility does not distinguish between beads with the same name in different molecules, so if one bead type is in more than one molecule type, its density will be averaged over all molecule types it appears in. If one requires densities specific to certain molecules containing the same bead types, the `-x` option can be used to first run the utility without one molecule type and then rerun it, excluding the other molecule type. Thus, two output files (per axis) are generated, each missing densities from the specified molecule types.

Note that this utility assumes orthogonal box with constant side lengths; in case of triclinic box and/or varying box size, undefined behaviour may occur, i.e., the utility may crash or freeze, and any results will not be reliable.

Usage: `DensityBox <input> <width> <output> [options]`

| Mandatory arguments | |
|---|---|
| `<input>` | input coordinate file (either `vcf` or `vtf` format) |
| `<width>` | width of each bin of the distribution |

| | |
|---|---|
| `<output>` | three output files with automatic `-x.rho`, `-y.rho`, and `-z.rho` endings |
| Options | |
| `-x <mol name(s)>` | exclude specified molecule type(s) (i.e., do not calculate density for beads in molecules `<mol name(s)>`) |
| Other options (see the beginning of Chapter 3) | |
| `-st`, `-e`, `-sk`, `-i`, `--help`, `--verbose`, `--silent`, `--version` | |

Format of output files:
1) `<output>` – bead densities; one file per x-, y-, and z-axis
- first line: AnalysisTools version
- second line: command used to generate the file
- third line: column headers
  - first is the centre of each bin (governed by `<width>`); i.e., if `<width>` is 0.1, then the centre of bin 0 to 0.1 is 0.05, centre of bin 0.1 to 0.2 is 0.15, etc.
  - the rest are for the calculated data: each column corresponds to the number density of the specified bead type
- the rest of the file are data lines

## 3.8 Info

`Info` reads an input structure file, prints system composition to standard output, and can optionally write the (possibly modified) system to a structure file of any supported format.

By default, the utility prints a summary of bead types, molecule types, counts, and simulation box dimensions. Using `--verbose` additionally prints per-bead and per-molecule details. When a secondary structure or coordinate file (`-i` or `-c` option, respectively) is present as well, `--verbose` also prints the initial composition of each input before the final merged result.

### Supplementing system information

A secondary structure file supplied via the `-i` option fills in bead type properties (charge, mass, radius) that are missing from the primary file, matching bead types by name. If both files share molecule types with identical names and bead type order, the secondary file can also supply bond types, angles, and related topology data absent from the primary file; note that all bead type parameters must match exactly for this to work.

The `--chbt` option goes further: instead of only filling in missing values, it remaps the bead types inside each molecule to those defined in the secondary file. Molecules are matched by name and bead count; molecule types with mismatched bead counts are skipped.

For `vtf` structure files, bead type identity is determined by bead name alone by default. The `--detailed` flag additionally uses charge, mass, and radius to

distinguish bead types, so beads sharing a name but differing in other properties are assigned distinct types. For example, three `atom` lines with the same name but two different masses and one with no mass normally produce a single bead type; with `--detailed` they become three separate types (one with undefined mass).

### Coordinates

Structure file formats that natively embed coordinates (`data`, `lammpstrj`, `xyz`) include them automatically; for `vtf` files, structure and coordinates may differ (not all beads in the structure block/file have to be in the coordinate block/file), so to read coordinates from a `vtf` file, the coordinate file (e.g., the same `vtf` file) must be included via the `-c` option. The `-st` option selects which timestep of the coordinate file to use (default: the first). When coordinates are available, the system is pruned to contain only the beads present in that timestep. If no coordinates are read, the system is not pruned, and all coordinates in the output structure file (`-o` option) will be 0's.

### Restructuring the system

The `--frag` option splits topologically disconnected molecules (molecules whose bonds do not form a single connected graph) into separate fragment molecule types named `<orig_name>_1`, `<orig_name>_2`, etc. Single-bead fragments are demoted to unbonded beads.

The `--mol` option converts all unbonded beads into one-bead molecules, using the bead type name as the molecule type name. This is useful when the output is to be processed by utilities that only operate on molecules (e.g., `Aggregates`).

### Output structure file

When `-o` is specified, `Info` writes the final system to a structure file; the format is inferred from the file extension. Writing to `itp` or `pdb` is not supported. Only one timestep of coordinates is written.

For `vtf`/`vsf` output, `-def` specifies the bead type for the `atom default` line; the default is the bead type with the greatest number of unbonded beads.

For LAMMPS `data` output, `--mass` collapses bead types that differ only in charge into a single LAMMPS atom type in the `Mass` section while preserving per-atom charges in the `Atoms` section. The `-ebt` option appends a given number of extra dummy atom types (mass 1, absent from the `Atoms` section), useful for, e.g., SRP ghost particles that require pre-declared atom type slots.

### Library-based bead renaming

The `-lib` option points to a library directory from which bead type names are reassigned and DPD interaction parameters are looked up. The resulting interaction table is printed to standard output in addition to the system composition. When the output is a `FIELD` file (`-o` option), the interaction block is also appended to that file.

If molecule types in the input file are unnamed, `-sys` supplies a `system_info` file to assign names to them before library renaming. Without `-sys`, unnamed molecule types cause the renaming step to be skipped with a warning.

TODO: Describe the `system_info` file format, or cross-referenc the relevant section once written.

TODO: Add `-lib`/`-sys` examples to `Examples/Info`.

Usage: `Info <input> [options]`

| Mandatory argument | |
|---|---|
| `<input>` | input structure |
| **Options for input files** | |
| `-i <file> [type]` | secondary structure file supplying additional information |
| `-c <file>` | input coordinate file |
| `--detailed` | (`vtf` input only) identify bead types by name, charge, mass, and radius rather than name alone |
| **Options for system modification** | |
| `--chbt` | replace bead types in molecules using the `-i` file; molecules matched by name and bead count |
| `--frag` | split disconnected molecules into fragment molecule types; single-bead fragments become unbonded beads |
| `--mol` | convert unbonded beads into one-bead molecules |
| `--unique` | make all bead and molecule names unique |
| **Options for output file** | |
| `-o <file>` | output structure file (`itp` and `pdb` output not supported) |
| `-def <bead>` | (`vtf`/`vsf` output only) bead type for the `atom default` line (default: type with the most unbonded beads) |
| `--mass` | (`data` output only) define LAMMPS atom types by mass and print per-atom charges in the `Atoms` section |
| `-ebt <int>` | (`data` output only) number of extra dummy atom types to add |
| **Library options** | |
| `-lib <dir>` | library directory for bead type renaming and DPD interaction lookup |
| `-sys <file>` | `system\_info` file to name unnamed molecule types before library renaming |
| **Other options (see the beginning of Chapter 3)** | |
| `-ft`, `-st`, `-e`, `-sk`, `--verbose`, `--silent`, `--help`, `--version` | |

## 3.9 JoinSystems

This utility takes two existing systems and joins them into a single system that contains all beads from the two coordinate files.

The second system can be offset against the first one (`-off` option). In each direction, all beads in the second system can be moved by specified distance (only positive numbers are allowed) or moved so that the centre of that box side is in the centre of the first box's side (use `c` instead of a number as an argument). The final box size is defined as the smallest box that encompasses the two original boxes after the second one is moved. However, the final box size can be redefined via the `-b` option; the centre of the new box is aligned with the centre of the original 'smallest encompassing box'. Note that no check is done whether the bead coordinates are within the new box.

Figure 3.3 illustrates the above-described behaviour (the red and blue lines and balls represent the two original systems while the green lines and transparent balls

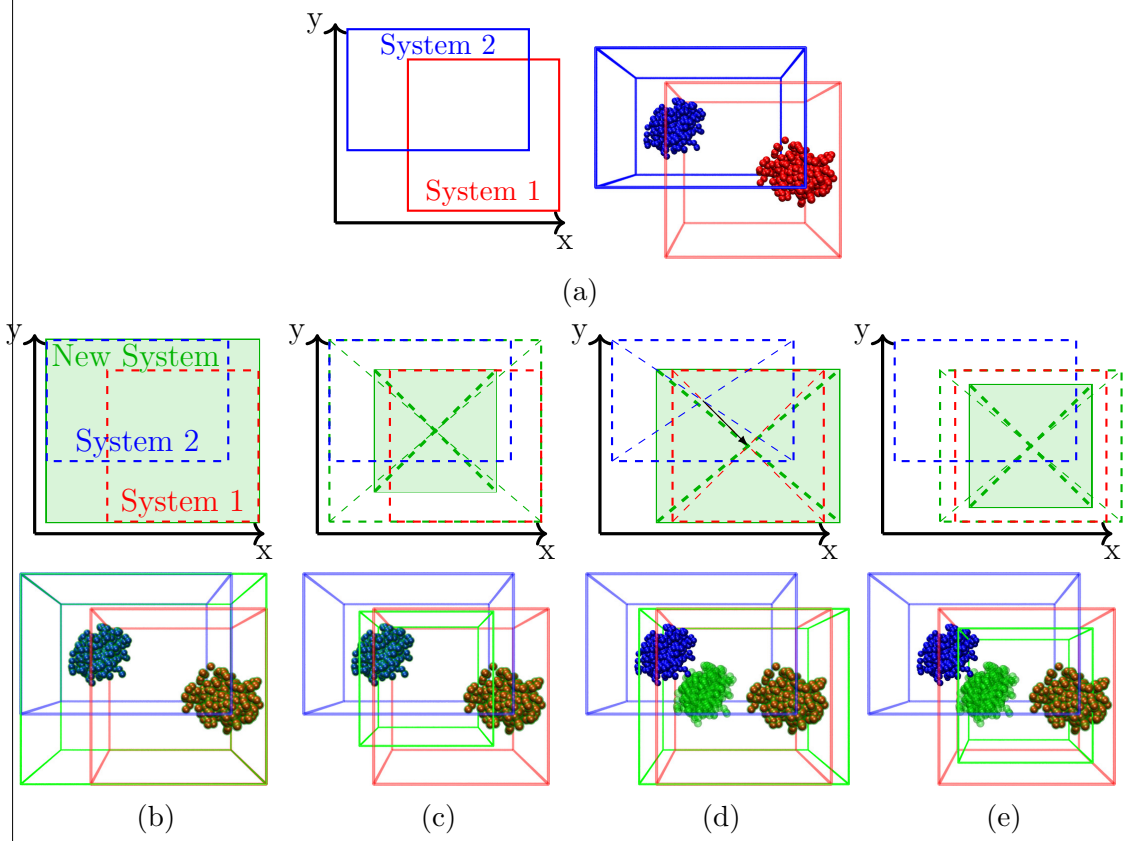



Figure 3.3: Schematic representation and corresponding snapshots illustrating `JoinSystems` behaviour. (a) The original systems: `System1.lammpstrj` (red) and `System2.lammpstrj` (blue) in the `Examples/JoinSystems` folder. The rest show examples of possible options (green rectangles and transparent balls represent the new systems): (b) no options; (c) `-b 20 20 20` option; (d) `-off c c c`; (e) `-b 20 20 20 -off c c c`, i.e., combining (c) and (d). These new systems are in the `Examples/JoinSystems` folder as `NewSystem\_\#.lammpstrj`, where `\#` is `b`, `c`, `d`, or `e`.

represent the new system). Figure 3.3b shows the simplest case when two systems are put into one larger system. In figure 3.3c, the final box size is also redefined; the positions of the beads is unchanged, only the box size changes. Note that the absolute bead coordinates remain unchanged only when the output coordinate file supports defining the bounds of the simulation box; e.g., in `lammpstrj` file, the lower and upper bounds of the box are defined while for `vtf` file, only the sidelengths of the box are specified. Should `vtf` be used as the output file, the saved coordinates would, therefore, move so the box's origin is in the coordinate beginning.

Using the `-off c c c` option in figure 3.3d moves the second system (the blue one), so that the centre of its box (in all three dimensions) lies on top of the centre of the first system's box (see the arrow in the schematic in figure 3.3d); coordinates of only the second system are adjusted, changing the relative coordinates of beads from the two original systems. If the `-off` and `-b` options are combined (as in figure 3.3e), the system created via the `-off` option is then assigned a new box size.

These examples are also provided in the `Examples/JoinSystems` directory.

The output file can be any file that supports coordinates, and an extra structure file can be created via the `-o` option. Note that if an output `vcf` coordinate file is used, a `vsf` structure file (with the same name except for the extension) is also created.

Usage: `JoinSystems <input1> <input2> <output> [options]`

| Mandatory arguments | |
|---|---|
| `<input1>` | first input coordinate file |
| `<input2>` | second input coordinate file |
| `<output>` | output file with the new system |

| Options | |
|---|---|
| `-off <x>\|c <y>\|c <z>\|c` | offset of the second system against the first (`c` to place it in the middle of the first system) |
| `-b <x> <y> <z>` | side lengths of the new simulation box |
| `-o <file>` | optional output structure file |

Other options (see the beginning of Utilities (chapter 3))
As two input coordinate/structure files are needed, some options have 1 or 2 appended to specify which file they correspond to

`-i1`/`-i2`, `-st1`/`-st2`, `--verbose`, `--silent`, `--help`, `--version`

## 3.10 Selected

This utility creates a new coordinate file. By default, the utility saves all bead and molecules types (basically transforming one coordinate file format into another), but there are options to fine-tune what is excluded or kept (`-bt`, `-mt`, `--keep`, `-cx/y/z`). There are also some basic manipulations for the coordinates from connecting molecules split via periodic boundary conditions or wrapping beads back into the box (`--join`, `--keep`) to scaling coordinates or moving particles (`-sc`, `-m`).

### Selecting bead and/or molecule types to exclude or keep

The `-bt` and `-mt` options select bead and molecule types, respectively. By default, the selected types are excluded (i.e., all other types are saved into the output file); the `--keep` option reverses the behaviour (i.e., only the selected types are saved).

Table 3.12: Interactions of `-bt`, `-mt`, and `--keep` options when used on a system containing bead types that are both unbonded and in molecules. For molecule types, the name represents initial composition (e.g., `ABC` comprises 1 bead of each of `A`, `B`, and `C` bead types), and the brackets show the final composition of the molecule (e.g., 1 (no `B`) for `ABC` molecule means only `A` and `C` beads remain in the molecule). For bead types, $a + b$ indicates unbonded beads ($a$) and beads in molecules ($b$).

| Options used on 'full' | Molecule types | | | Bead types | | |
|---|---|---|---|---|---|---|
| | ABC | AB | AC | A | B | C |
| full | 1 | 1 | 1 | 10+3 | 10+2 | 10+2 |
| `-bt B` | 1 (no B) | 1 (no B) | 1 | 10+3 | 0 | 10+2 |
| `-bt B -mt AB` | 1 (no B) | 0 | 1 | 10+2 | 0 | 10+2 |
| `-bt B -mt AB --keep` | 1 (only B) | 1 | 0 | 0+2 | 10+2 | 0 |

Table 3.12 shows possible interactions between the three options when some beads are unbonded and other beads of the same type are in a bond. The 'full' system is in the `Examples/Selected` directory as the `full.vtf` file.

### Constraining and manipulating coordinates

The `-cx`, `-cy`, and `-cz` options constrain beads to be saved to specified coordinate ranges (by default, the range is in fractions of the simulation box). The options accept multiples of number pairs for each axis; a bead coordinate must fall within at least one of the ranges, e.g., using `-cx 0 0.1 0.9 1` would save beads whose $x$ coordinates are within 0 to 10% or 90% to 100% of the box size in the x-direction. Adding, e.g., `-cy 0.4 0.6` would save only the beads that fulfil the `-cx` option as well as this one, i.e., that $y$ coordinate is between 40% and 60% of the simulation box.

The `-sc <float>` option scales all coordinates by dividing them via the specified number.

The `-m <x> <y> <z>` option moves all coordinates by the specified vector (by default, the vector is in fractions of simulation box).

If multiple options are used, they manipulate the coordinates in the order `-cx/y/z`, `-sc`, `-m`, i.e. first, coordinates are constrained, then scaled, and lastly moved. See figure 3.4 for an example: if all three options are specified in one command, it transforms the 'original' system in the direction of 'all options at once' with the beads ending up outside the box. This is equivalent to running three consecutive commands, first only with `-cx/y/z` (from 'original' system to the right), then with `-sc` (down), and lastly with `-m` (left). Figure 3.4 shows what happens when the

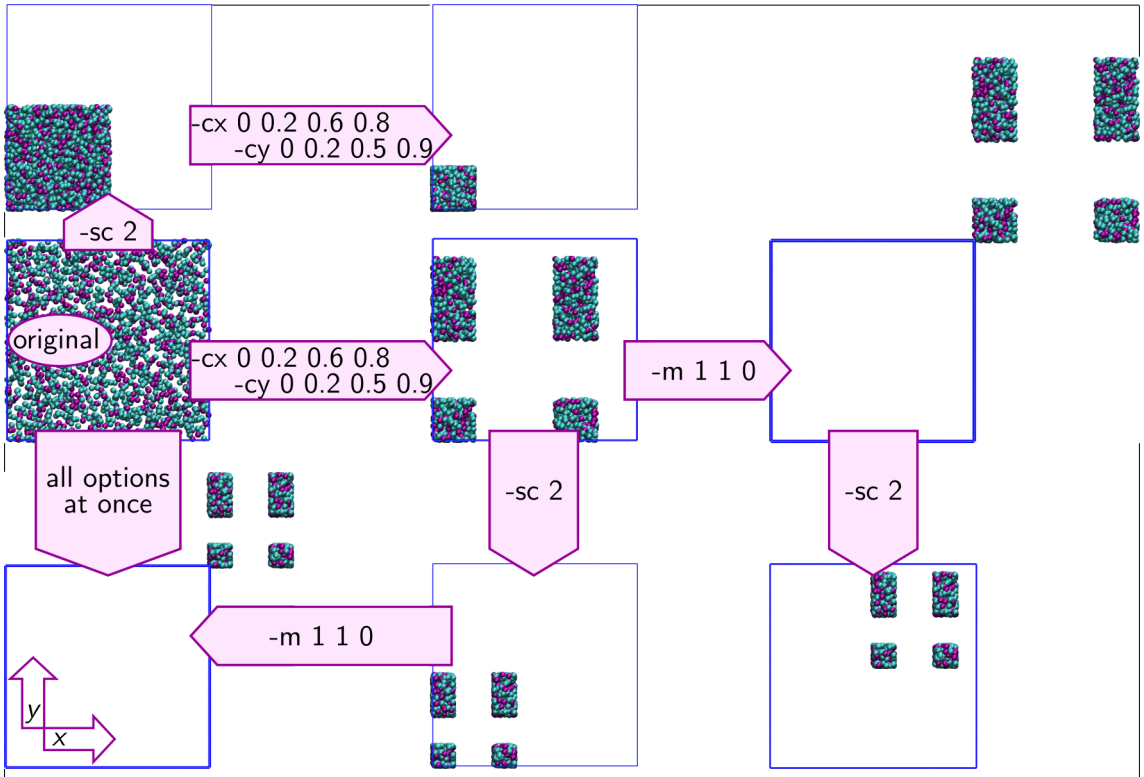



Figure 3.4: Some of the possible results when combining sequential runs of the `Selected` utility, namely (1) constraining $x$ and $y$ coordinates via `-cx 0 0.2 0.6 0.8 -cy 0 0.2 0.5 0.9`, (2) scaling (dividing) coordinates via `-sc 2`, and (3) moving coordinates via `-m 1 1 0`. The blue lines represent the simulation box (the varying line thickness is just an artefact of how the snapshots were created in `vmd`).

`Selected` commands are run in different order. These examples are also in the `Examples/Selected` directory.

As mentioned above, both `-cx/y/z` and `-m` options accept fractions of box size (i.e., numbers between 0 and 1) by default. To specify 'real' coordinates instead, use the `--real` option.

Note that neither of these options changes the size of the simulation box.

### Dealing with periodic boundary conditions

The input system may contain beads that are outside the simulation box. The `--wrap` option returns all beads into the simulation box...

Conversely, molecules can be disconnected due to the periodic boundary conditions with one part of the molecule on one side of the box and another part on the opposite side. The `--join` removes the periodic boundary conditions, making the molecule whole as well as stick out of the simulation box. The centre of mass of every 'joined' molecule is inside the simulation box.

If both options are specified, the coordinates are first wrapped via `--wrap` and then the molecules are 'joined' via `--join`.

Usage: `Selected <input> <output> <bead type(s)> [options]`

| Mandatory arguments | |
|---|---|
| `<input>` | input coordinate file |
| `<output>` | output coordinate file |
| **Options** | |
| `-bt <bead type>` | bead types to exclude |
| `-mt <mol type>` | molecule types to exclude |
| `--keep` | save only specified types instead of excluding them |
| `--join` | join molecules by removing periodic boundary conditions |
| `--wrap` | wrap simulation box (i.e., apply periodic boundary conditions) |
| `-n <int(s)>` | save only specified timesteps |
| `--last` | save only the last step |
| `-sc <float>` | divide all coordinates by given value |
| `-m <x> <y> <z>` | add specified vector to all coordinates (`-sc` is applied first) |
| `-cx n*2*<float>` | constrain x-coordinates to specified dimension; multiple pairs possible |
| `-cy n*2*<float>` | same as `-cx` but with y-coordinates |
| `-cz n*2*<float>` | same as `-cx` but with z-coordinates |
| `--real` | use real coordinates instead of fractions of box size (for `-cx/y/z` and `-m` options) |
| **Other options (see the beginning of Chapter 3)** | |
| `-st`, `-e`, `-sk`, `-i`, `--verbose`, `--silent`, `--help`, `--version` | |

## 3.11 Surface

Surface utility determines the average surface of structures such as membranes or polymer brushes and calculates the surface areas.

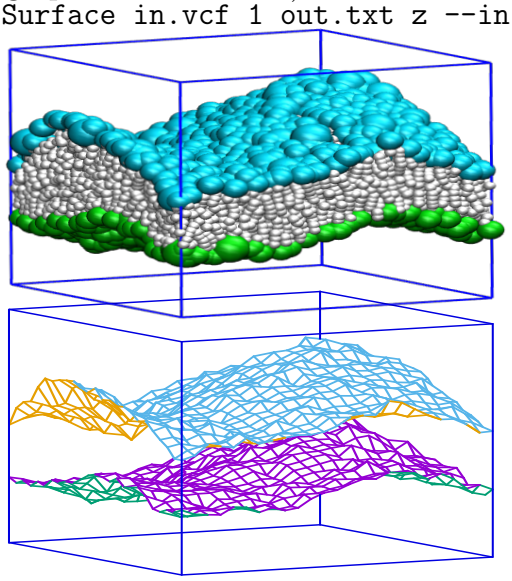
The utility cuts the plane perpendicular to the chosen axis into squares of size `<width>*<width>` and then finds the beads with the highest and lowest coordinates along the chosen axis whose other two coordinates fall into the square for each square on the plane. By default, it searches the chosen axis for the beads with the highest coordinate in the interval $\langle 0, (boxlength)/2\rangle$ and with the lowest coordinate in the interval $\langle (box\ length)/2; (box\ length)\rangle$, i.e., it assumes something such as a polymer brush on each wall. If `--in` option is used, it searches for a layer structure inside the box, i.e., it searches for the bead with the lowest and highest coordinates in the whole box, $\langle 0; (box\ length)\rangle$ (this mode finds surfaces for something like a bilayer).

Note that the utility does not determine any structures for now, therefore any molecules outside the brush/membrane can be recognized as a part of the surface (`-bt` and `-m` can be sometimes used to eliminate this problem).

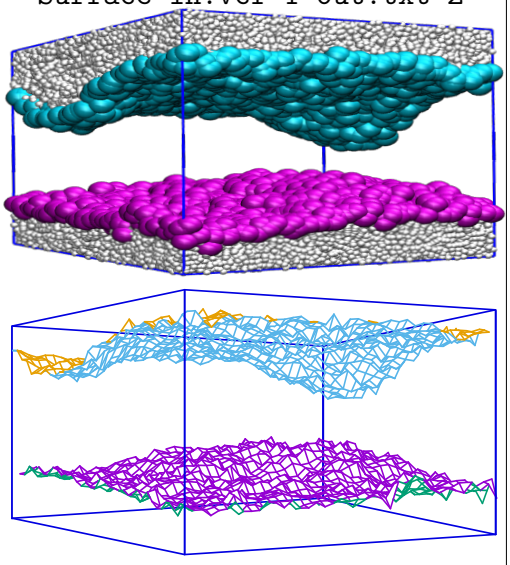
By default, all bead types and all molecule types are used, but using `-bt` and `-m` options, only specified bead and/or molecule types can be used. This is particularly useful when, e.g., other molecules solubilize inside the brush/membrane, leaving some molecules in the solution.

Following are two examples of usage with and without `--in` option (on top are snapshots with coloured balls representing the detected surface and on the bottom are graphs of the surface):

`Surface in.vcf 1 out.txt z --in`



`Surface in.vcf 1 out.txt z`



Usage:

Usage: `Surface <input> <width> <surf.txt> <area.txt> <axis> [options]`

| Mandatory arguments | |
|---|---|
| `<input>` | input coordinate file (either `vcf` or `vtf` format) |
| `<width>` | side length of each square |
| `<surf.txt>` | output file with average surface |
| `<area.txt>` | output file with per-timestep areas |
| `<axis>` | direction in which to determine the surface: `x`, `y`, or `z` |

| Options | |
|---|---|
| `--in` | start from the box centre instead of from its edges |
| `--bonded` | use beads in molecules for surface recognition |
| `-bt <name(s)>` | use specified bead types for surface recognition |

Other options (see the beginning of Chapter 3)

`-st`, `-e`, `-sk`, `-i`, `--verbose`, `--silent`, `--help`, `--version`

Format of output files:

1) `<surf.txt>` – 3D coordinates
   - first line: AnalysisTools version
   - second line: command used to generate the file
   - third line: column headers
   - first two numbers represent the centre of each square (being the coordinates on the sliced up plane); i.e., if `<width>` is 1, then the centre of the first square is 0.5 0.5, the centre of the second one is 0.5 1.5, etc.
   - the second two numbers are coordinates of the surface in the third dimension, i.e., along the chosen axis

- data lines follow

2) `<area.txt>` – per-timestep areas (calculated as a sum of triangles)
- first line: AnalysisTools version
- second line: command used to generate the file
- third line: column headers
  - first is simulation timestep
  - the rest are areas of the top, bottom, and central surfaces
- data lines follow
- second to last line: column headers for overall averages
- last line: the overall averages